Vitala Health Analysis Platform

Complete Technical Architecture, Repository Replication & Vercel Deployment Manual

1. Project Executive Overview

Vitala is an enterprise clinical physiological intelligence platform engineered to perform comprehensive biometric assessment, AI symptom evaluation, real-time geospatial medical facility discovery, and longitudinal health activity tracking. The system adheres to strict clinical validation thresholds and operates with high-availability serverless and client-side processing fallbacks.

Core Module	Clinical & Functional Capability
Predict Engine	Evaluates 9 vitals (Heart Rate, SpO2, BP, Temp in Fahrenheit °F, Resp Rate, Glucose, ECG, Thyroid, Telemetry). Generates 0-100 Health Rate and Risk Level.
Symptom Analysis	Natural language diagnostic evaluation for non-monitored conditions, offering urgency classification, self-care steps, and specialist recommendations.
Nearby Care & Doctors	100% authentic GPS location discovery querying OpenStreetMap Overpass & Nominatim APIs for 24/7 trauma hospitals, clinics, and direct Google Maps navigation.
Health History	Client-segregated activity archiving with statistical score metrics, historical report inspection modals, and JSON export.
Juli Assistant	Ultra-concise precision AI health companion (under 35 words, 1-2 bullet points, zero disclaimers).
Authentication	Encrypted credentials storage in local storage with smooth mode transition and persistent session management.

2. Complete Technology Stack & Libraries

Component Layer	Technology / Framework	Version & Purpose
Frontend UI Library	React (React-DOM)	v19.2.8 — Component lifecycle & reactivity
Build & Dev Tool	Vite	v8.2.2 — High-performance bundling & HMR
Styling Engine	Tailwind CSS	v4.3.3 (@tailwindcss/vite plugin)
Icon System	Lucide React	v1.38.0 — Clinical vector iconography
AI Diagnostic Models	Google Gemini Flash	gemini-3.6-flash REST API endpoint
Geospatial Discovery	OpenStreetMap Overpass / Nominatim	Live GIS nodes & reverse geocoding
Storage & Persistence	Browser LocalStorage (db.js)	Segregated JSON encrypted records
Deployment Platform	Vercel Serverless	Zero-config SPA & API route handlers

3. Step-by-Step Project Development Workflow

Step 1: Scaffolding and Environment Setup

- Initialized Vite project with React 19 template and configured Tailwind CSS v4 pipeline.
- Configured strict clean typography, custom blueprint color palettes (#08162B, #0F766E, #0F2747), and responsive viewports.

Step 2: Authentication & Database Engine

- Built src/services/db.js with member registration, password hashing/storage, activity logging, and active session distribution.
- Created centered frosted authentication card with password visibility toggles, mail/lock icons, and instant tab transitions.

Step 3: Clinical Predict Engine & Visual Charts

- Enforced strict Fahrenheit standard (°F) for body temperature (97.0°F - 99.0°F normal bracket).
- Built interactive Range Tracks, vertical Variance Bar Chart (h-72), and Status Distribution Donut Chart.

Step 4: Authentic Geospatial Care Locator

- Integrated browser navigator.geolocation with Nominatim reverse-geocoding.
- Executed live Overpass QL queries fetching real hospital nodes, emergency tags, telephone numbers, and operating hours.

Step 5: Health History & Juli Assistant

- Implemented user history auto-recording on every analysis run, filter tabs, modal inspectors, and JSON export.
- Built ultra-concise Juli companion connected to Gemini with strict 35-word constraints.

4. How to Copy / Duplicate Repository to Another GitHub Account

To duplicate this project to a new or different GitHub account or repository without copying Git commit history:

```
# 1. Clone the original repository to your machine
git clone https://github.com/Eswar5048/vitala-health-analysis.git new-vitala-project

# 2. Enter the project directory
cd new-vitala-project

# 3. Create a brand-new repository on your target GitHub account (e.g. at https://github.com/new)

# 4. Remove the old remote origin
git remote remove origin

# 5. Add your new target GitHub repository as origin
git remote add origin https://github.com/YOUR_NEW_USERNAME/YOUR_NEW_REPO_NAME.git

# 6. Rename branch to main and push all code
git branch -M main
git push -u origin main
```

5. Step-by-Step Git Commands to Push Code

When you make updates or modifications to the codebase, use the standard Git workflow:

```
# Step 1: Check modified and untracked files
git status

# Step 2: Stage all changes
git add .

# Step 3: Create a meaningful descriptive commit
git commit -m "feat: enhance clinical metrics and update visual charts"

# Step 4: Ensure current branch is main
git branch -M main

# Step 5: Push commits to GitHub
git push origin main
```

6. How to Host on Vercel (Step-by-Step)

- **Step 1**: Log into **https://vercel.com** using your GitHub account credentials.
- **Step 2**: Click **Add New...** → **Project** (or visit **https://vercel.com/new**).
- **Step 3**: Locate vitala-health-analysis from your repository list and click **Import**.
- **Step 4**: Under **Project Name**, enter a unique name (e.g., vitala-health or vitala-app).
- **Step 5**: Vercel will automatically detect **Framework Preset: Vite** (Root Directory: ./, Build Command: vite build, Output: dist).
- **Step 6**: Expand the **Environment Variables** dropdown before clicking Deploy.
- **Step 7**: Enter the two required keys (detailed in Section 7 below) and click **Deploy**.
- **Step 8**: Vercel builds the app in ~30 seconds and outputs your live production URL (e.g., https://vitala-health.vercel.app).

7. Environment Variables Configuration (Secure Token Representation)

To protect your security, never commit raw secret keys to public Git repositories. Use the exact environment variable key names shown below in your Vercel Project Settings or local .env file:

Environment Variable Name	Target Module	Token Value Format
GEMINI_HEALTH_API_KEY	Predict & Symptom Analysis Engine	<YOUR_GEMINI_HEALTH_API_KEY_TOKEN>
GEMINI_JULI_API_KEY	Juli Clinical Assistant	<YOUR_GEMINI_JULI_API_KEY_TOKEN>

```
# Local .env template (Create in root project directory)
GEMINI_HEALTH_API_KEY=
GEMINI_JULI_API_KEY=
```

Security & Production Note:
• **Serverless Execution:** In production on Vercel, requests to /api/* are routed through serverless functions in the api/ directory, protecting your API tokens from browser exposure.
• **Client Fallbacks:** If offline or self-hosted statically, the platform falls back to native mathematical calculations and direct OpenStreetMap GIS lookups seamlessly.